# AI가 만든 코드를 읽고 수정하기

## 비개발자 창업가의 웹 개발 학습

구조를 찾고, 실행을 따라가고, 바꿀 위치를 판단하는 연습

# 내가 이 수업을 배우는 이유

## AI가 코드를 생성했을 때

1 어떤 파일이 화면의 어느 부분을 만드는지 찾기

2 버튼을 누른 뒤 데이터가 어떻게 바뀌는지 설명하기

3 수정할 위치를 고르고, 실행 결과로 확인하기

# 수업의 전체 흐름

| | | |
|---|---|---|
| 01-03 | HTML | 내용과 입력칸의 구조 |
| 04-08 | CSS와 Git | 배치, 반응형, 변경 기록, Bootstrap |
| 10-14 | JavaScript | 값, 조건, 함수, 배열과 객체 |
| 15-21 | 동작과 데이터 | 이벤트, 서버 통신, 저장, 모듈 |
| 27-29 | React | 컴포넌트, props, state, Effect |

# HTML은 화면의 내용과 구조를 정한다

브라우저는 HTML을 받아 읽습니다.

form과 input으로 값을 받습니다.
label로 입력칸 이름을 연결합니다.

main과 article로 구조를 구분합니다.

```
<main>
  <h1>오즈 카페</h1>
  <form action="/signup" method="post">
    <label for="email">이메일</label>
    <input id="email" name="email"
           type="email" required />
  </form>
</main>
```

# CSS는 같은 구조의 모양을 바꾼다

선택자로 수정할 대상을 찾습니다.

여백과 배치를 정하고
화면 폭에 따라 규칙을 더합니다.

색이 그대로라면 적용 규칙을 봅니다.

```
.menu-list {
  display: grid;
  gap: 16px;
  grid-template-columns: 1fr;
}

@media (min-width: 768px) {
  .menu-list {
    grid-template-columns: 1fr 1fr;
  }
}
```

# Git은 수정 전후를 기록한다

| | | |
|---|---|---|
| 1 | 변경 확인 | 내가 바꾼 파일과 줄을 비교 |
| 2 | 기록할 변경 선택 | git add |
| 3 | 로컬 기록 | git commit |
| 4 | 원격 업로드 | git push |

# Bootstrap은 준비된 스타일을 사용한다

클래스 이름으로 배치와 모양을
빠르게 적용합니다.

모달처럼 열고 닫는 기능에는
연결된 JavaScript도 필요합니다.

```
<div class="row">
  <main class="col-12 col-lg-8">
    본문
  </main>
  <aside class="col-12 col-lg-4">
    최근 글
  </aside>
</div>
```

# JavaScript는 값을 받아 조건에 따라 계산한다

변수에 값을 담습니다.
함수에 값을 전달합니다.
조건과 반복으로 계산합니다.

return은 결과를 호출한 곳으로
돌려줍니다.

```
function getPrice(age) {
  if (age < 13) {
    return 5000;
  }
  return 10000;
}

const price = getPrice(12);
console.log(price); // 5000
```

# 배열과 객체로 목록을 처리한다

객체는 항목 하나의 정보입니다.
배열은 여러 항목의 목록입니다.

filter는 조건에 맞는 항목,
map은 바꾼 결과를 모읍니다.

```
const menus = [
  { name: "커피", price: 4000 },
  { name: "빵", price: 3500 }
];

const names = menus
  .filter(m => m.price <= 3500)
  .map(m => m.name);

// 결과: ["빵"]
```

# 이벤트는 사용자 행동과 함수를 연결한다

DOM에서 화면 요소를 찾고
클릭할 때 실행할 함수를 연결합니다.

연결할 때는 함수 자체를 넘깁니다.

```
const button =
  document.querySelector("#add");
const result =
  document.querySelector("#result");
let count = 0;

button.addEventListener("click", () => {
  count += 1;
  result.textContent = count;
});
```

# 서버 응답을 기다린 뒤 화면에 반영한다

await로 비동기 결과를 기다립니다.
응답 상태와 본문을 나눠 읽습니다.

404 응답도 상태 검사가 필요합니다.
실패는 catch에서 안내합니다.

```
async function loadPost(url) {
  try {
    const res = await fetch(url);
    if (!res.ok) {
      throw new Error("불러오기 실패");
    }
    const post = await res.json();
    return post;
  } catch (error) {
    console.log(error.message);
  }
}
```

# 함수는 다른 함수를 만들어 돌려줄 수 있다

콜백은 나중에 실행할 함수입니다.

클로저는 함수가 만들어진 곳의
변수를 함께 기억합니다.

버튼마다 다른 구매 동작을
연결합니다.

```
function makeBuyer(name, price) {
  return () => machine.buy(name, price);
}

button.addEventListener(
  "click",
  makeBuyer("물", 800)
);
```

# 저장과 파일 분리로 데이터를 관리한다

localStorage는 새로고침 뒤에도 필요한 값을 복원하는 데 씁니다.

import와 export는 역할별로 나눈 파일을 연결합니다.

```
// 저장: 배열을 문자열로
localStorage.setItem(
  "readIds", JSON.stringify([1, 3])
);

// 다른 파일의 함수를 사용
import pickMenu from "./menu.js";
import { menus } from "./menu.js";

const menu = pickMenu(menus);
```

# React는 컴포넌트에 값을 전달해 화면을 만든다

개발 서버로 앱을 실행합니다.
화면을 컴포넌트로 나눕니다.

props를 바꾸면 같은 컴포넌트를
다른 내용으로 사용할 수 있습니다.

```
function MenuCard({ name, price }) {
  return (
    <article>
      <h3>{name}</h3>
      <p>{price}원</p>
    </article>
  );
}

<MenuCard name="커피" price={4000} />
```

# state를 바꾸면 새 값으로 화면을 그린다

state는 렌더 사이에 기억할
값입니다.
set 함수로 다음 값을 요청합니다.

Effect는 탭 제목처럼
React 바깥의 상태를 맞춥니다.

```
const [count, setCount] = useState(0);

function handleAdd() {
  setCount(c => c + 1);
}

useEffect(() => {
  document.title = `담은 메뉴 ${count}개`;
}, [count]);
```

# 담기 버튼 한 번의 실행 흐름

1. MenuCard의 담기 버튼을 누른다
2. props로 받은 onAdd 함수를 호출한다
3. App의 handleAdd가 count 갱신을 요청한다
4. 새 count가 Header에 전달되어 숫자가 바뀐다
5. Effect가 브라우저 탭 제목을 맞춘다

# 직접 수정할 때는 데이터가 바뀌는 곳을 찾는다

## 한 번에 두 개 담기

```
setCount(c => c + 2);
```

## 숫자를 0으로 초기화하기

```
<button onClick={() => setCount(0)}>초기화</button>
```

화면의 개수와 탭 제목이 함께 바뀌는지 확인합니다.

# AI 코드에서 확인할 질문

무엇이 이 코드를 실행시키는가?

어떤 값이 어디에서 바뀌는가?

바뀐 값은 화면의 어디에 나타나는가?

수정 전후의 결과를 어떻게 비교할 것인가?

# 발표 범위와 자료 출처

다운로드 폴더의 수업자료 PDF 28개
Notion 데일리 과제 23개와 DAY29 시작 ZIP

DAY09 및 DAY22-26은 수업 본문을 확보하지 못했습니다.
DAY30 이후는 이번 발표 범위에 포함하지 않았습니다.

Notion 데일리 과제
https://app.notion.com/p/41654c6415d781a0b2bace55624c0eed